

CSS-

Inline CSS

Inline CSS involves directly embedding CSS styles within HTML elements using the `style` attribute. This method applies styles to individual elements, overriding external stylesheets.

Internal CSS

Internal CSS involves embedding CSS rules directly within an HTML document. This is achieved by using the `<style>` tag, typically placed inside the `<head>` section of the HTML file.

External CSS

External CSS involves writing CSS rules in a separate file (with a `.css` extension) and then linking that file to your HTML document. This approach promotes cleaner code, easier maintenance, and reusability of styles across multiple web pages.

Cascading Order in CSS (Simple Explanation)

Cascading Order means:

When multiple CSS rules apply to the same element, which one will win?

Browsers follow a priority system to decide this.

CSS Priority Rules (Highest → Lowest)

1. `!important` (Highest Priority)

- Overrides everything

```
p {  
  color: red !important;  
}
```

2. Inline CSS

- Written directly inside HTML

```
<p style="color: blue;">Hello</p>
```

3. Specificity (Very Important)

More specific selector = higher priority

Example:

```
p { color: red; }           /* low */  
.class { color: blue; }    /* higher */  
#id { color: green; }     /* highest */
```

Priority:

```
#id > .class > element
```

4. Source Order (Last Rule Wins)

If specificity is same → last written CSS applies

```
p { color: red; }  
p { color: blue; } /* this will apply */
```

Final Priority Order (Shortcut Trick)

☞ "IISSS" Rule

1. **I** → Important (!important)
2. **I** → Inline CSS
3. **S** → Specificity
4. **S** → Source Order

Properties and Values in CSS-

What is property?

A **property** is *what you want to change* in an element.

Examples:

- color → text color
- font-size → text size
- background-color → background

What is a Value?

☞ A **value** is *how you want it to look*.

Examples:

```
color: red;  
color: #00FF00;  
color: rgb(0, 255, 0);
```

Here:

- color = property
- red / #00FF00 / rgb(...) = values

Property + Value = Declaration

☞ A **declaration** is a combination of property and value.

```
color: blue;    ✓ This is one declaration
```

What is a Selector in CSS?

🔗 A **selector** is used to **target HTML elements** so you can apply styles to them.

Simple selectors in CSS are the basic building blocks for targeting HTML elements you want to style. They directly select elements based on their name (like `p`, for paragraphs), ID (using `#`), or class (using `.`). These selectors are straightforward and provide a fundamental way to apply styles to specific parts of your webpage

Types of Selectors (Most Important)

1. 🏠 Element Selector

Targets HTML tags

```
p {  
  color: blue;  
}
```

🔗 All `<p>` tags will be blue

2. 🏷️ Class Selector

Targets elements with a class

```
.box {  
  background-color: yellow;  
}  
<div class="box"></div>
```

🔗 Uses `.` (dot)

3. 🏠 ID Selector

Targets a unique element

```
#header {  
  color: red;  
}  
<h1 id="header"></h1>
```

🔗 Uses `#`

🔗 Should be unique

4. 🌐 Universal Selector

Targets everything

```
* {  
  margin: 0;  
}
```

5. 📦 Group Selector

Apply same style to multiple elements

```
h1, p {  
  color: green;  
}
```

6. Descendant Selector

Targets elements inside another

```
div p {  
  color: blue;  
}
```

 All <p> inside <div>

What is a Declaration in CSS?

 A **declaration** is a single line of CSS that defines:

- **what to style (property)**
- **how to style it (value)**

CSS Rules

CSS rules are the fundamental building blocks of CSS stylesheets. Each rule specifies how particular HTML elements should be styled. A rule consists of a selector, which identifies the element(s) to be styled, and a declaration block, which contains one or more property-value pairs that define the styles to be applied.

Comments-

/* This is a CSS comment */[Simple Selectors](#)

Attribute selectors-

Attribute selectors in CSS are used to select elements based on their attributes or attribute values for more precise styling.

Selector	Meaning
[attr]	Has attribute
[attr="value"]	Exact match
[attr^="val"]	Starts with
[attr\$="val"]	Ends with
[attr*="val"]	Contains

Font families-

Font families in CSS specify a list of fonts that the browser uses to display text, choosing the first available one.

Example:

```
p {  
  font-family: Arial, Helvetica, sans-serif;  
}
```

Font style-

Font style in CSS specifies whether text should be normal, italic, or oblique.

Font Size-

Font size controls how big or small the text appears on a webpage.

Font Weight-

Font weight refers to the thickness or boldness of characters .

Font shorthand-

Font shorthand is a CSS property used to set multiple font-related properties in a single declaration.

```
p {  
  font: italic bold 16px/1.5 Arial, sans-serif;  
}
```

Google Fonts-

Google Fonts is a free library of fonts provided by [Google Fonts](https://fonts.google.com/).

<!DOCTYPE html>

<html>

<head>

<link href="https://fonts.googleapis.com/css2?family=Roboto&display=swap" rel="stylesheet">

<style>

body {

font-family: 'Roboto', sans-serif;

}

</style>

</head>

<body>

<h1>Hello World</h1>

</body>

</html>

Font variant-

Font variant in CSS is used to control different stylistic variations of text, such as small-caps.

```
p {  
font-variant: small-caps;  
}
```

Element Selectors-

Element selectors in CSS target HTML elements directly by their tag name. EXAMPLE-<p>, <h1>.

CLASS SELECTOR- (.)

ID SELECTOR- (#)

UNIVERSAL SELECTOR- (*)

GROUPING SELECTOR - Grouping selectors allow you to apply the **same styles to multiple elements at once**.

```
h1, p, div {  
color: blue;  
}
```

DIRECTION-

The direction property in CSS is used to control the text flow direction (LTR or RTL), though using the HTML `dir` attribute is recommended for better semantics.

Text alignment-

Text alignment in CSS is used to control the horizontal positioning of text within an element using values like left, right, center, and justify.

Text decoration-

Text decoration in CSS is used to style text with lines such as underline, overline, or line-through.

Text transform-

Text transform in CSS controls how text is capitalized (uppercase, lowercase, etc.), regardless of the original HTML text.

Important Background Properties

```
selector {  
  
background-image: url("image.jpg");  
  
}
```

1. **background-size** → controls image size

```
background-size: cover;      /* fills entire screen */
background-size: contain;    /* fits image inside */
```

2. **background-repeat** → controls repeating

```
background-repeat: no-repeat;
```

3. **background-position** → sets position

```
background-position: center;
```

4. **background-attachment** → scroll behavior

```
background-attachment: fixed
```

5. **background gradient**—Colors flow in a straight line.

```
background: linear-gradient(to right, red, blue)
```

☞ Direction options:

```
to right
to left
to top
to bottom
```

☞ With angle:

```
background: linear-gradient(45deg, red, yellow);
```

Opacity in CSS

Opacity controls how transparent an element is.

- `opacity: 1` → fully visible (default)
- `opacity: 0` → completely invisible
- `opacity: 0.5` → semi-transparent

These are three ways to define colors in CSS — RGB, HEX, and HSL

1. **RGB (Red, Green, Blue)**

☞ Defines color using intensity of **Red, Green, Blue (0–255)**

```
color: rgb(255, 0, 0);    /* red */
color: rgb(0, 255, 0);    /* green */
color: rgb(0, 0, 255);    /* blue */
```

☞ With transparency:

```
color: rgba(255, 0, 0, 0.5); /* 50% transparent red */
```

- ✓ Easy to understand
- ✓ Good for dynamic values (JS)

◆ 2. HEX (Hexadecimal)

☞ Uses **base-16 values (0–9, A–F)**

```
color: #ff0000; /* red */
color: #00ff00; /* green */
color: #0000ff; /* blue */
```

☞ Short form:

```
color: #f00; /* same as #ff0000 */
```

- ✓ Most commonly used
- ✓ Compact format

◆ 3. HSL (Hue, Saturation, Lightness)

☞ More **human-friendly** way to adjust colors

```
color: hsl(0, 100%, 50%); /* red */
color: hsl(120, 100%, 50%); /* green */
color: hsl(240, 100%, 50%); /* blue */
```

- **Hue (0–360)** → color type
- **Saturation (0–100%)** → intensity
- **Lightness (0–100%)** → brightness

☞ With transparency:

```
color: hsla(0, 100%, 50%, 0.5);
```

- ✓ Best for designing & tweaking shades
- ✓ Easy to make lighter/darker colors

1. RGBA (Red, Green, Blue, Alpha)

☞ Based on RGB + transparency

```
color: rgba(255, 0, 0, 1); /* fully visible red */
color: rgba(255, 0, 0, 0.5); /* semi-transparent red */
color: rgba(255, 0, 0, 0); /* invisible */
```

✓ Values:

- Red, Green, Blue → 0–255
- Alpha → 0–1

◆ Example

```
.box {
  background-color: rgba(0, 0, 0, 0.5);
}
```

```
color: white;
}
```

☞ Creates a dark transparent background (very common in UI)

🔗 2. HSLA (Hue, Saturation, Lightness, Alpha)

☞ Based on HSL + transparency

```
color: hsla(0, 100%, 50%, 1); /* red */
color: hsla(0, 100%, 50%, 0.5); /* transparent red */
```

✓ Values:

- Hue → 0–360
- Saturation → %
- Lightness → %
- Alpha → 0–1

CSS Box Model

The **CSS Box Model** defines how every HTML element is structured as a rectangular box.

Each element consists of **4 parts**:



1. Content

- Actual data (text, image, etc.)
- Size controlled by:

width, height

2. Padding(top, right, bottom, left)

- Space **inside the box**, around content

```
padding: 10px;
```

3. Border

- Surrounds padding + content

```
border: 2px solid black;
```

4. Margin

- Space **outside the box**

```
margin: 20px;
```

Padding vs Margin:

Property	Meaning
Padding	Space inside (content → border)
Margin	Space outside (border → other elements)

Padding	Margin
Padding can be defined as the inner space of the element.	Margin can be described as the outer space of an element.
Padding cannot be set to auto.	You can set the margin to auto.
It can't be set to any negative value.	It can be set to negative or any float number value.
Styling of an element affects the padding such as background color.	Margin is not affected by styling of other elements.

Border

Border = visible outline around an element

```
div {  
  border: 2px solid black;  
}
```

☞ This means:

- 2px → thickness
- solid → style
- black → color

[Outline in CSS](#)

Outline is a line drawn **outside the border** of an element.

CSS UNITS-

Unit Type	Examples	Behavior
Absolute	px, cm, in	Fixed size
Relative	em, rem, %	Depends on parent
Viewport	vw, vh	Depends on screen

BLOCK VS INLINE VS INLINE-BLOCK

Property	Block	Inline	Inline-Block
New line	✓ Yes	✗ No	✗ No
Width/Height	✓ Yes	✗ No	✓ Yes
Space taken	Full width	Content only	Content + set size

Visibility in CSS

visibility controls whether an element is **visible or hidden**, without removing it from the layout.

Visibility vs Display

Property	visibility: hidden	display: none
Visible?	✗ No	✗ No
Space occupied?	✓ Yes	✗ No
Layout affected?	✗ No	✓ Yes

Display: none

display: none completely removes an element from the webpage.

Key Points:

- Element is **not visible**
- It takes **no space**
- It is **removed from document flow**
- Other elements **shift and adjust automatically**
-

```
• position: sticky;  
• position: fixed;  
• position: absolute;  
• position: relative;  
• position: static;  
• */  
• position: absolute;  
• top: 50px;  
• left: 50px;
```

- ☞ If question asks “**which removes element from flow?**”
Answer: **absolute & fixed**
- ☞ If asks “**which keeps space?**”
Answer: **static, relative, sticky**

Pseudo-Elements in CSS

Pseudo-elements let you style **specific parts of an element** or create content that doesn't exist in HTML.

☞ They use **double colon ::**

1. `::before`

Adds content **before** an element

```
p::before {  
  content: "☞ ";  
  color: red;  
}
```

☞ 2. `::after`

Adds content **after** an element

```
p::after {  
  content: " ✓";  
  color: green;  
}
```

☞ 3. `::first-letter`

Styles the **first letter**

```
p::first-letter {  
  font-size: 30px;  
  color: blue;  
}
```

☞ 4. `::first-line`

Styles the **first line**

```
p::first-line {  
  font-weight: bold;  
}
```

☞ 5. `::selection`

Styles selected text

```
::selection {  
  background: yellow;  
  color: black;  
}
```

z-index controls which element appears on top **when elements overlap.**

```
.box1 {  
  position: relative;  
  z-index: 1;  
}
```

```
.box2 {  
  position: relative;  
  z-index: 10;  
}
```

☞ **.box2 will be on top**
Because **10 > 1**

Stacking Context with Parent & Child (a.k.a. “stacking containers”)

When people say “**stacking containers**”, they usually mean **parent and child stacking contexts**—how layers are grouped and compared.

🔑 Core Idea

- A **parent** can create a **stacking context (container)**
- All its **children are stacked inside it only**
- A child’s `z-index` is **limited to its parent’s context**

CSS Specificity

CSS **Specificity** decides **which CSS rule wins** when multiple rules target the same element.

📖 It’s like a **priority system (weight system)**

📊 Specificity Order (Most Important 🔥)

Selector Type	Priority
<code>!important</code>	🔥 Highest
Inline styles	1000
ID (<code>#id</code>)	100
Class (<code>.class</code>)	10
Element (<code>div</code>)	1

Pseudo-Elements in CSS (Detailed)

🔑 Definition

Pseudo-elements let you style **specific parts of an element** or insert content **that doesn’t exist in HTML**.

📖 Syntax:

```
selector::pseudo-element {  
  property: value;  
}  
  
<!DOCTYPE html>  
<html>  
<head>  
<style>  
p::before {
```

```

    content: "☞ ";
    color: red;
}

p::after {
    content: " ✓";
    color: green;
}
</style>
</head>

<body>

<p>Hello World</p>

</body>
</html>

```

[Pseudo-Classes in CSS \(Clear + Detailed\)](#)

Pseudo-classes are used to style elements based on their **state, position, or condition**—without changing HTML or using JavaScript.

☞ Syntax:

```

selector:pseudo-class {
    property: value;
}

```

✓ Uses **single colon :**

```

<!DOCTYPE html>

<html>

<head>

<style>

button {

    background: gray;

    padding: 10px;

}

```

	Selector Type	Priority
	button:hover {	
	background: blue;	
	}	
	button:active {	
	background: red;	
	}	
	/* Input focus */	
	input:focus {	
	border: 2px solid green;	
	}	
	</style>	
	</head>	
	<body>	
	<button>Click Me</button>	
	<input type="text" placeholder="Click me">	
	</body>	
	</html>	

What is Flexbox?

Flexbox (Flexible Box Layout) is a layout system used to **align, arrange, and distribute space** among elements inside a container.

☞ Works in **one direction**:

- Row (horizontal)
- Column (vertical)

✦ Example (ROW)

```
.container {
  display: flex;
  flex-direction: row;
  justify-content: center; /* horizontal */
  align-items: center;     /* vertical */
}
```

✦ Example (Column)

```
.container {
  display: flex;
  flex-direction: column;
  justify-content: center; /* vertical */
  align-items: center;     /* horizontal */
}
```

Feature	Block	Inline	Inline-Block
New line	✓ Yes	✗ No	✗ No
Width/Height	✓ Yes	✗ No	✓ Yes
Space taken	Full width	Content only	Content + set size

[li:nth-child\(2n\)](#)

☞ Selects **even elements** (2,4,6...)

Flexbox axes:

- **Main axis** → horizontal (left ↔ right)
- **Cross axis** → vertical (top ↑ bottom)